

Algoritmos Exactos y Metaheurísticas: Tarea 1

Benjamín Aceituno, Diego Banda

Profesor: Víctor Reyes

Ayudante: Felipe Gutiérrez

4 de Abril, 2025

Índice

1. Introducción	3
1.0.1. Datos de enunciado	3
2. Desarrollo	5
2.1. Modelo del problema	5
2.1.1. Conjuntos	5
2.1.2. Variables y Dominio	5
2.1.3. Constantes y Dominio	5
2.1.4. Restricción	5
2.1.5. Función objetivo	6
2.1.6. Espacio de búsqueda	6
2.2. Búsqueda completa	6
2.2.1. Variables	7
2.2.2. Restricción	7
2.2.3. Función solución	8
2.3. Heurística	10
2.3.1. Variables	10
2.3.2. función heuristic()	10
2.3.3. Función solución	11
3. Experimentación	12
4. Análisis	13
5. Conclusión	17
6. Bibliografía	18

1. Introducción

Se entrega un enunciado con un problema a desarrollar, este problema de título '*La temida influenza*' dice que se acerca el invierno y, para anticiparse a problemas de colapso, se quieren construir centros de vacunación para toda la población, cada centro tiene capacidad para la propia comuna donde se instala y las comunas adyacentes. Instalar un centro de vacunación en una comuna tiene un costo asociado a esa comuna, se deben buscar las comunas en las cuales instalar los centros de vacunación con el objetivo de minimizar los costos lo máximo posible, pero cumpliendo con cubrir todas las comunas presentes (la propia y sus adyacentes).

1.0.1. Datos de enunciado

El enunciado entrega ciertos datos los cuales serán importantes tener en cuenta para posteriormente realizar los experimentos, ya que con estos datos se realiza la ejecución de los códigos y experimentos/comparaciones.

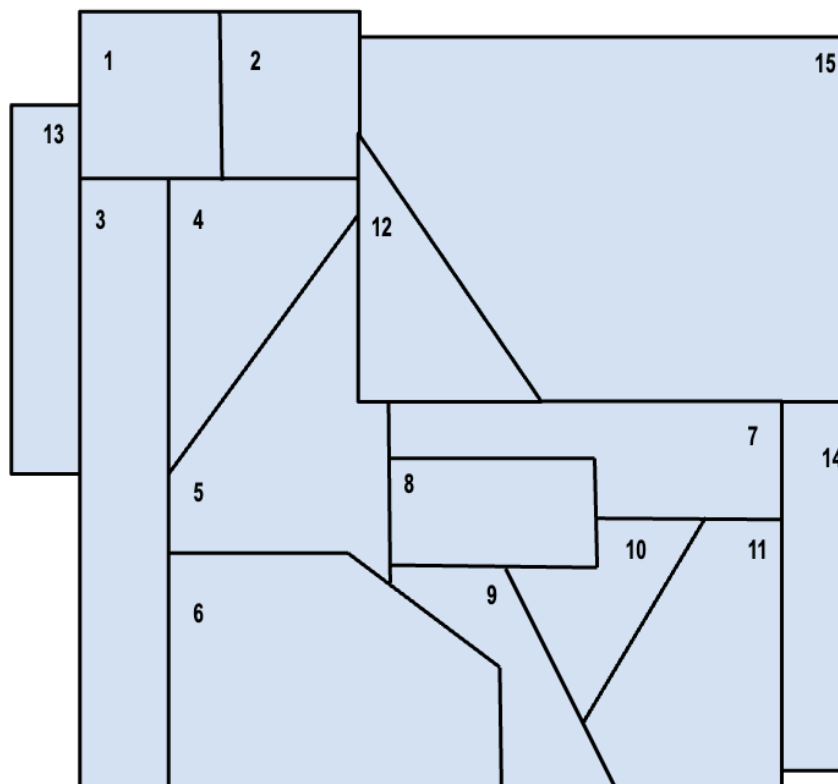


Figura 1: Comunas de enunciado

A partir de este mapa podemos deducir la siguiente tabla con adyacencias:

Comuna	Comunas Adyacentes	Comunas	Comunas Adyacentes
1	2, 3, 4, 13	9	5, 6, 8, 10, 11
2	1, 4, 12, 15	10	7, 8, 9, 11
3	1, 4, 5, 6, 13	11	7, 9, 10, 14
4	1, 2, 3, 5, 12	12	2, 4, 5, 7, 15
5	3, 4, 6, 7, 8, 9	13	1, 3
6	3, 5, 9	14	7, 11, 15
7	5, 8, 10, 11, 12, 14, 15	15	2, 7, 12, 14
8	5, 7, 9, 10		

Tabla 1: Comunas y sus comunas adyacentes

También, se deben tener en cuenta los costos; estos definirán gran parte de la solución, ya que el costo es lo que se busca minimizar.

Comuna	Costo	Comuna	Costo
1	60	9	80
2	30	10	70
3	60	11	50
4	70	12	90
5	130	13	30
6	60	14	30
7	70	15	100
8	60		

Tabla 2: Costos para cada comuna

2. Desarrollo

2.1. Modelo del problema

Para esta tarea, se desarrolló un modelo de optimización con restricciones (COP) para el problema de la influenza en las comunas de la región Brisketiana, considerando variables, dominios, restricciones y función objetivo. En este problema se busca minimizar el costo asociado a construir centros de vacunación en las comunas de la región.

2.1.1. Conjuntos

Para el modelo, necesitamos decretar los conjuntos con los cuales vamos a definir este problema y sus distintas restricciones.

- **Conjunto de comunas (C):** Representa todas las comunas en la región donde se necesita cobertura.

$$C = \{1, 2, 3, \dots, 15\}$$

2.1.2. Variables y Dominio

Debemos determinar una variable para los límites determinados del problema; en este caso, necesitamos una variable que contemple si queremos poner un centro de vacunación en una comuna o no.

- **Binaria de comuna:** Representa si se pone o no un centro de vacunación en una comuna i .

$$X_i = \{0, 1\}$$

2.1.3. Constantes y Dominio

En este problema tenemos constantes que están asociadas al costo de construcción de cada centro de vacunación.

- **Costo de construcción:** Representa el costo de construir un centro de vacunación en una comuna i .

$$C_i = \{60, 30, 60, 70, 130, 60, 70, 60, 80, 70, 50, 90, 30, 30, 100\}$$

2.1.4. Restricción

Este problema tiene una sola restricción, y es que se debe satisfacer toda la demanda de centros de vacunación, es decir, que cada comuna debe tener un centro, ya sea en sí misma o en alguna de sus comunas adyacentes.

$$\sum_{i \in A_j \cup \{j\}} X_i \geq 1, \quad \forall j$$

- Donde A_j son las comunas adyacentes a j
 - Por ejemplo: $A_4 = \{1, 2, 3, 5, 12\}$ o $A_8 = \{5, 7, 9, 10\}$
- $A_j \cup \{j\}$ es para que la restricción considere a la misma comuna j y no solo las adyacentes.
 - Por ejemplo, al considerar A_4 sería $\{1, 2, 3, 5, 12\} \cup \{4\}$

2.1.5. Función objetivo

La función objetivo de este problema busca minimizar el costo de construcción de los centros de vacunación, satisfaciendo la condición de que todas las comunas estén cubiertas a lo menos por un centro de vacunación.

$$\min \sum_i X_i C_i$$

2.1.6. Espacio de búsqueda

El espacio de búsqueda de este problema está dado por todas las posibles combinaciones de comunas en las que se puede instalar un centro de vacunación. Cada comuna tiene dos espacios posibles.

- **0**: No se instala un centro de vacunación en la comuna.
- **1**: Se instala un centro de vacunación en la comuna.

Esto nos da que el espacio de búsqueda para las 15 comunas está dado por:

$$2^{15} = 32,768$$

2.2. Búsqueda completa

Las técnicas de búsqueda completa son aquellas que iteran alrededor de todo el espacio de búsqueda, llegando el 100 % de los casos a la mejor solución posible, este tipo de búsqueda tiene como ventaja eso, que siempre llegará a la mejor solución o terminará su ejecución en caso de no existir, sin embargo, computacionalmente son muy costosos, llevando a que si tu espacio de búsqueda es muy grande, se demore grandes cantidades de tiempo en llegar a una solución. En este caso, como se vio anteriormente, el espacio de búsqueda es tan solo de 32,768, lo cual para un computador de hoy en día no es excesivamente grande.

La búsqueda consiste en asignarle un valor a la variable que sea su turno en esa iteración y avanzar hacia el siguiente nodo, esto se realiza a través de la recursión, se le asigna un valor a la variable y se llama a la función nuevamente, pero con esta variable modificada. De esta manera se crea un árbol, en el cual, los nodos no-hoja son soluciones incompletas, pero que

pueden entregar información de hacia donde se dirige la solución, y los nodos hoja, si es que cumplen con las restricciones, son una solución factible.

En este trabajo se utiliza una técnica llamada *Backtracking Cronológico*, como se mencionó anteriormente, los nodos no-hoja entregan información de hacia donde está encaminándose la solución. En este contexto, si la función objetivo de un nodo no-hoja es igual o mayor a la función objetivo de la mejor solución encontrada hasta el momento, no se sigue buscando por esa rama del árbol, ya que en ningún caso entregará una solución mejor que la encontrada.

2.2.1. Variables

El código se apoya principalmente en las siguientes variables:

1. Variable en Recursión

- ***vector<int> x***: Variable que se modifica en cada ciclo de la función recursiva, el índice i representa si se pone o no un centro de vacunación en la comuna $i + 1$.

2. Variables Globales

- ***int comunas***: Cantidad de comunas.
- ***vector<int> c***: Costos de cada comuna, el índice i es el costo de la comuna $i + 1$.
- ***vector<vector<int>> ady***: Las comunas adyacentes de cada comuna, i es el índice del vector que contiene las comunas adyacentes de la comuna $i + 1$.
- ***vector<int> sol***: Vector que contiene la mejor solución encontrada hasta el momento, tiene el mismo formato que x .
- ***int minFound***: Se guarda el mínimo valor de función objetivo encontrado hasta el momento de una solución factible, se inicializa en el número máximo soportado por los tipos de dato entero.

3. Datos

- ***int nodeCount***: Cuenta la cantidad de nodos actuales.
- ***long startTime***: Guarda el tiempo de inicio para posteriormente calcular el tiempo en el que se encuentra cada solución.
- ***int solutionsFound***: Cuenta la cantidad de soluciones factibles encontradas.

2.2.2. Restricción

Para la restricción presente se utiliza una función llamada *constraintSatisfaction*, la cual recibe el vector x que contiene la posible solución encontrada, itera sobre cada comuna verificando que exista un centro en sí misma o en alguna de sus comunas adyacentes, en caso de encontrar una comuna que no cumpla, retorna false, en caso de terminar de iterar sobre todas las comunas y no encontrar alguna que no cumpla, retorna true.

Input: x : Vector de enteros

Output: **true** si se satisface la restricción para cada comuna, **false** en caso contrario

```

for  $i \leftarrow 0$  to comunas do
     $aux \leftarrow 0$ ;
    if  $x[i] = 1$  then
         $aux \leftarrow 1$ ;
    end
    for  $j \leftarrow 0$  to tamaño de ady[ $i$ ] do
        if  $x[ady[i][j] - 1] = 1$  then
             $aux \leftarrow aux + 1$ ;
        end
    end
    if  $aux < 1$  then
        return false;
    end
end
return true;

```

Algorithm 1: constraintSatisfaction

2.2.3. Función solución

Para la búsqueda de una solución se llama a la función *solve*, inicialmente recibe un vector x lleno, desde $i = 0$ hasta $i = comunas - 1$, con 0 e i y *objectiveFunction* inicialmente son iguales a 0, i es el índice de la comuna en la que el nodo se encuentra evaluando actualmente.

Esta función utiliza la recursión para generar el árbol de búsqueda, en cada llamado a la función se le entrega el vector x con la decisión de poner un centro en la comuna i o no (modificando con un 0 o 1 la posición i), $i+1$ y *objectiveFunction* (en caso de agregar un centro en la comuna i se le suma a *objectiveFunction* el costo de poner un centro en la comuna $i+1$). A la vez, cada nodo nuevo le agrega 1 a la cantidad de nodos, si la función objetivo actual es mayor o igual que la mejor encontrada (la menor), se termina de buscar a través de esa rama, esto es porque si la solución ya es igual o mayor que la mejor encontrada, no se encontrará una solución mejor a través de esa rama. Si i es igual a comunas, significa que ya se llegó al nodo hoja y es una solución, se verifica que esta sea una solución factible y que su función objetivo sea menor a la encontrada actualmente, en caso de ser así, se reemplaza la mejor solución encontrada por la nueva, se suma uno a la cantidad de soluciones encontradas y se imprime la solución junto a los datos asociados.

Input: x : Vector de enteros, i : Índice actual, *objectiveFunction*: Valor de la función objetivo

Output: Actualiza la mejor solución encontrada

$nodeCount \leftarrow nodeCount + 1$;

if *objectiveFunction* $\geq$ *minFound* **then**

return;

end

if $i = comunas$ **then**

if *constraintSatisfaction*(x) **and** *objectiveFunction* $<$ *minFound* **then**

$sol \leftarrow x$;

$minFound \leftarrow objectiveFunction$;

$solutionsFound \leftarrow solutionsFound + 1$;

printSolve(*objectiveFunction*);

end

return;

end

solve($x, i + 1, objectiveFunction$);

// Caso en que no se agrega un centro en la comuna i

$x[i] \leftarrow 1$;

// Caso en que sí se agrega un centro en la comuna i

solve($x, i + 1, objectiveFunction + c[i]$);

Algorithm 2: *solve*

Para imprimir la solución se utiliza la función *printSolve*, la cual enseña por pantalla la solución encontrada, la lista de comunas que minimizan la solución, la función objetivo, los nodos actuales y tiempo en los que se encontró la solución.

Output: Imprime información de la solución actual

$endTime \leftarrow getTime()$;

Imprimir "---- SOLUCION NUMERO " *solutionsFound* "----";

Imprimir "Lista de comunas en las que se instalan centros de vacunación";

for $i \leftarrow 0$ **to** tamaño de *sol* **do**

if $sol[i] = 1$ **then**

 Imprimir $i + 1$;

end

end

Imprimir salto de línea;

Imprimir "Funcion Objetivo (calidad de solucion): " *minFound*;

Imprimir "Nodos actuales: " *nodeCount*;

Imprimir "Tiempo transcurrido: " $endTime - startTime$ "ms";

Algorithm 3: *printSolve*

2.3. Heurística

Una heurística, en palabras simples, es una función que bajo la información del momento toma una decisión, en este caso se aplicará una heurística de selección de variable, la cual se ejecutará al inicio para ordenar de cierta manera las comunas, así, se busca encontrar mejores primera solución y bajar la cantidad de de nodos utilizados y el tiempo en encontrar la mejor solución. En este caso se utiliza una heurística minimizando bajo la función:

$$heuristicVal = \sum_{i=0}^{n-1} \sum_{j=0}^{m_i-1} 1$$

Donde:

- **n:** *ady.size()*.
- **m:** *ady[i].size()*.

Lo que hará nuestra heurística es ordenar las comunas (sus índices) desde la que tiene más comunas adyacentes primero hacia la que tiene menos, esto ya que una comuna con más adyacencias está más presente en las distintas restricciones de demanda, consiguiendo una mejor solución inicial y por consecuencia bajar la cantidad de nodos y tiempo en encontrar el minimo global.

2.3.1. Variables

Para añadir un orden bajo la heurística mencionada se agregaron dos variables globales nuevas que son necesarias:

- ***vector<int>* orden:** Se añaden los índices de las comunas ordenadas bajo la heurística.
- ***vector<int>* heuristicVal:** El valor de la heurística de la comuna $i + 1$.

2.3.2. función heuristic()

La función *heuristic* se utiliza al inicio para ordenar las variables y luego hacer la búsqueda correspondiente, en esta función primero se llena el vector *heuristicVal* y luego se utiliza mergeSort para ordenar el vector *orden*.

Output: Inicializa los vectores *orden* y *heuristicVal*, y los ordena

for $i \leftarrow 0$ **to** *comunas* - 1 **do**

orden.push_back(i);

heuristicVal.push_back(|ady[i]|);

// Guarda el número de vecinos de la comuna i

end

mergeSort(0, comunas - 1) // Ordena los valores heurísticos

Algorithm 4: heuristic

2.3.3. Función solución

En cuanto a la función solución se debe hacer una pequeña modificación, para así utilizar la selección de variable correspondiente al nodo en el que se encuentra.

Input: x : Vector de enteros, i : Índice actual, *objectiveFunction*: Valor de la función objetivo

Output: Actualiza la mejor solución encontrada

$nodeCount \leftarrow nodeCount + 1$;

if *objectiveFunction* $\geq$ *minFound* **then**

return;

end

if $i = comunas$ **then**

if *constraintSatisfaction*(x) **and** *objectiveFunction* $<$ *minFound* **then**

$sol \leftarrow x$;

$minFound \leftarrow objectiveFunction$;

$solutionsFound \leftarrow solutionsFound + 1$;

 printSolve();

end

return;

end

solve($x, i + 1, objectiveFunction$);

// Caso en que no se agrega un centro en la comuna i

$x[orden[i]] \leftarrow 1$;

// Caso en que sí se agrega un centro en la comuna $orden[i]$

solve($x, i + 1, objectiveFunction + c[orden[i]]$);

Algorithm 5: solve 2

Se realiza un cambio a la hora de asignar 1 a la comuna correspondiente, ahora utilizando el vector *orden* para el índice de donde poner el centro de vacunación.

Inicialmente se probó con otras heurísticas para la selección de variable, sin embargo estas no mostraron resultados tan buenos como la seleccionada, esto puede ser por los datos del ejemplo e igual funcionar mejor en otros contextos, algunas de estas probadas son:

- **Menor costo:** Minimizar el costo de poner un centro en la comuna i

$$heuristicVal = c[i]$$

- **Cociente entre costo y comunas adyacentes:**

$$heuristicVal = \frac{c[i]}{ady[i].size()}$$

3. Experimentación

La experimentación y ejecución de los códigos se realizó a través de una computadora macbook air m1 con las siguientes características:

- **Procesador:** Chip M1 de apple (8 núcleos).
- **Memoria RAM:** 8 GB.
- **Memoria:** SSD 256 GB.

En cuanto a la experimentación se ejecutaron ambos códigos, ambos imprimen cada solución encontrada e información extra la cual podemos interpretar para saber qué tan costoso fue encontrar esa solución. A la hora de ejecutar el ambos códigos, estos enseñan la siguiente información:

N° solución	Comunas con centros de vacunación	Función objetivo	Nodos actuales	Tiempo transcurrido (ms)
1	9, 12, 13, 15	300	166	646
2	9, 12, 13, 14	230	168	733
3	4, 9, 13, 14	210	1098	2781
4	2, 9, 13, 14	170	1862	4197

Tabla 3: Experimentación datos sin heurística

N° solución	Comunas con centros de vacunación	Función objetivo	Nodos actuales	Tiempo transcurrido (ms)
1	2, 6, 10, 13, 14	220	345	987
2	2, 6, 8, 13, 14	210	385	1145
3	2, 9, 13, 14	170	884	2123

Tabla 4: Experimentación datos con heurística

Estos outputs nos entregan valorpreciado para entender el funcionamiento de ambos códigos, primero se puede observar que los dos convergen en la misma solución, pero estas tienen distintos tiempo transcurrido y cantidad de nodos, esto puede deberse a varios factores.

4. Análisis

Para el análisis, se realiza una comparación entre los resultados obtenidos con dos técnicas de resolución del problema COP presentado en la tarea, una utilizando backtracking cronológico sin heurística y la otra con la incorporación de una heurística de selección de variable. El objetivo es examinar de manera profunda cómo varían los costos computacionales, como los nodos explorados y el tiempo de ejecución, mientras que la calidad de solución entregada debe ser la misma ya que a pesar que una analiza más nodos que otra, ambas siguen siendo técnicas de búsqueda completa.

Primero se analizará las soluciones obtenidas por la técnica completa sin la implementación de una heurística, donde se registró su tiempo, el valor de la función objetivo, los nodos explorados y las comunas seleccionadas por la solución. Para la técnica completa sin heurística tenemos que el programa en *C++* nos entrega 4 soluciones óptimas y cada una de estas selecciona 4 comunas para implementar los centros de vacunación.

El tiempo que tardó el algoritmo en realizar el proceso de exploración y selección en las dos primeras soluciones es relativamente corto y la cantidad de nodos explorados también, esto indica un costo computacional bajo y que el algoritmo se ejecuta de manera eficiente, sin embargo, tenemos que estas soluciones no ofrecen un valor que minimiza los costos de instalación de centros de vacunación, valores más altos en la función objetivo indican una calidad de la solución baja, que no minimiza la función. Esto se debe a que el algoritmo de backtracking explora múltiples combinaciones posibles de manera exhaustiva, las soluciones factibles encontradas al inicio pueden no ser el óptimo global y quizá estar bastante alejadas de él. En las primeras etapas, el algoritmo puede encontrar soluciones rápidas que no minimizan eficazmente el costo total, ya que sigue evaluando nuevas combinaciones sin ninguna heurística que guíe la búsqueda hacia las soluciones más eficientes.

Como se mencionó, aunque el algoritmo es capaz de generar múltiples soluciones viables, se evidencia que aquellas que se obtienen con menor costo computacional no necesariamente representan las mejores soluciones desde el punto de vista de la Función Objetivo. Véase que las primeras soluciones tienen valores de F.O. significativamente más altos, lo que indica un mayor costo asociado a la instalación de los centros de vacunación. Esto contradice el objetivo principal del problema, que es minimizar dicho costo.

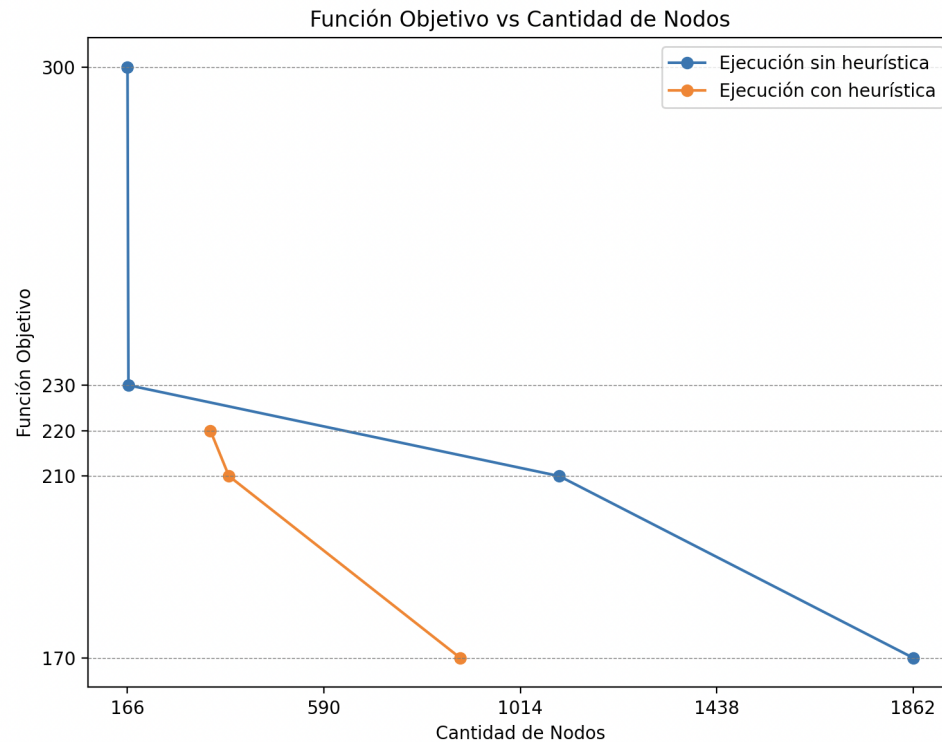


Figura 2: Función objetivo v/s cantidad de nodos para ejecución sin heurística y con heurística

En la figura 2 podemos ver que la ejecución sin heurística muestra un comportamiento muy distinto a la con heurística, debido a que si bien, las primeras ejecuciones sin heurística son de un costo computacional bajo, estas se disparan al seguir buscando distintas soluciones más cercas de la función objetivo, generando una pendiente más plana que sugiere que el algoritmo necesita recorrer muchas más combinaciones para encontrar mejores soluciones, lo que lo hace menos eficiente. Véase como la cantidad de nodos en el valor que minimiza la función objetivo es muy distintas entre cada una.

Este comportamiento es característico de los algoritmos de backtracking sin ningún tipo de guía inteligente, al explorar el espacio de soluciones de forma exhaustiva y sin priorización, tienden a encontrar soluciones rápidamente, pero sin asegurar que estas sean las más cercanas al óptimo global. Aunque se garantice eventualmente la exploración de todas las posibilidades, el recorrido puede ser ineficiente.

Esto cambia cuando se implementa la heurística de selección en el algoritmo de búsqueda completa, el proceso se transforma significativamente. La heurística actúa como un mecanismo que guía la búsqueda hacia regiones más prometedoras del espacio de soluciones, evaluando primero aquellas combinaciones que tienen mayor probabilidad de producir un costo bajo. Esto se traduce en una mejora considerable tanto en la calidad de las soluciones obtenidas como en la eficiencia computacional en etapas avanzadas del algoritmo.

Esto también se ve reflejado en el tiempo de ejecución, donde podemos ver que el uso de una heurística mejora significativamente el tiempo de ejecución, ya que permite encon-

trar soluciones de menor costo explorando menos nodos. Por otro lado, el backtracking sin heurística realiza una exploración más extensa, aumentando el tiempo de cómputo y retrasando la obtención de una solución óptima. Véase figura 3.

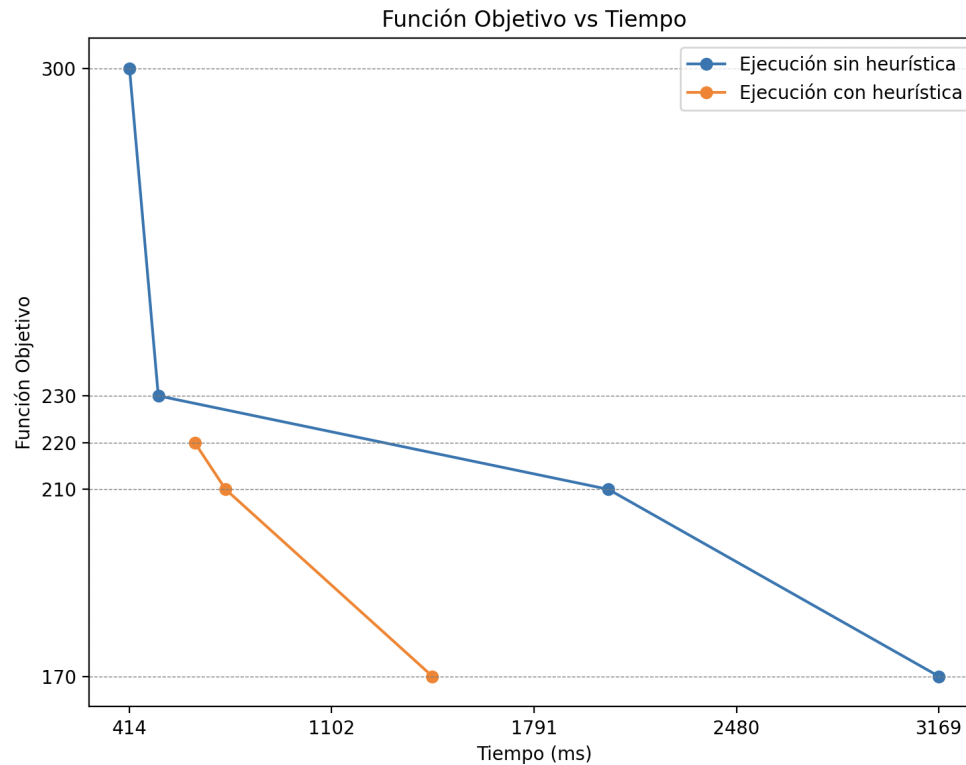


Figura 3: Función objetivo v/s tiempo en ms para ejecución sin heurística y con heurística

En general, se puede ver que el tiempo de cómputo y la cantidad de nodos explorados crecen de forma similar, aunque hay algunas pequeñas variaciones. Esto se debe a que el tiempo de ejecución está fuertemente influenciado por la cantidad de nodos procesados, lo cual sucede en un algoritmo de Backtracking. Además, con la heurística aplicada, la relación se vuelve más estable, lo que indica que optimiza la exploración y mantiene una relación más predecible entre los nodos y el tiempo. Véase figura 4.

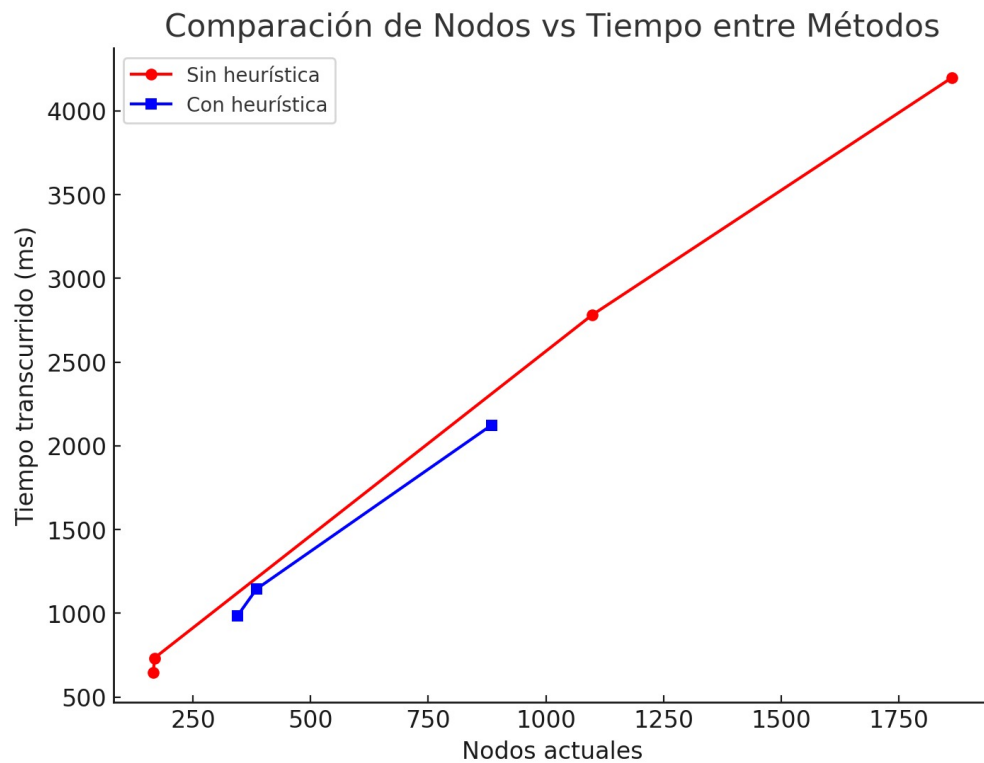


Figura 4: Nodos ejecutados v/s Tiempo de ms para ejecución sin heurística y con heurística

Por lo tanto, se concluye que la búsqueda completa con heurística no solo mejora el resultado final, al reducir efectivamente el costo de implementación de los centros de vacunación, sino que también contribuye a una exploración más inteligente y eficiente del espacio de soluciones.

5. Conclusión

En conclusión, los algoritmos de búsqueda completa son útiles para encontrar la solución global del problema que se le entregue, sin embargo, se debe tener presente que estos algoritmos son increíblemente ineficientes, por lo tanto, si el espacio de búsqueda es lo suficientemente grande, estos algoritmos tardarán muchísimo y quizá no sean la mejor opción si se quiere una solución rápida.

Por otro lado, se compararon dos formas de algoritmos de búsqueda completa, una sin selección de variable y otra con selección de variable, y se observó claramente la gran diferencia que genera el tener una heurística de selección de variable, bajando el costo computacionalmente en aproximadamente un 50 %, esto debido a la buena solución inicial encontrada y poda de nodos no prometedores.

Finalmente, se puede decir que es buena idea utilizar un algoritmo de búsqueda completa cuando los espacios de búsqueda a ejecutar son relativamente pequeños, llevando a que el algoritmo no tardará demasiados minutos en llegar a la mejor solución posible, en caso contrario, si el espacio de búsqueda es demasiado grande, quizá sea recomendable utilizar otras técnicas para llegar a soluciones factibles. También es importante destacar el cambio ocasionado al agregar la heurística de selección de variable, ya que baja considerablemente el thrashing y el costo computacional de ejecutar una técnica completa, ayudando con espacios de búsqueda que sean más grande y que tomen más tiempo.

6. Bibliografía

1. Reyes, V. (2025) *Algoritmos Exactos y Metaheurísticas* [Diapositivas]. Universidad Diego Portales.
2. JosephSan — Coder. (2023, 9 de enero). *C++ — Calcular tiempo de ejecución de cualquier operación/función con C++ y time.h (CPU Runtime)* [Video]. Youtube. <https://www.youtube.com/watch?v=uyG1Ju8nRiA>